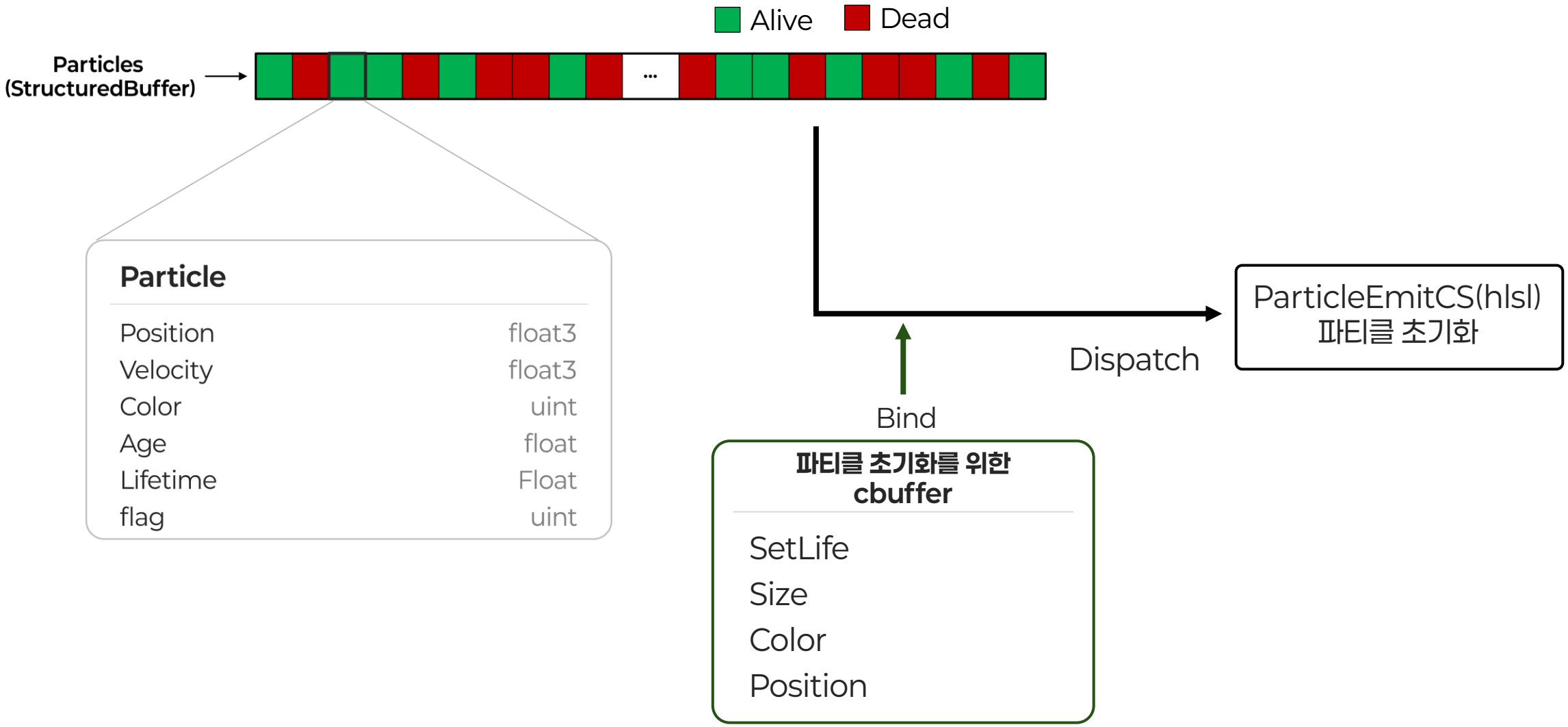
GPU 기반 파티클 시스템 구현 및 최적화

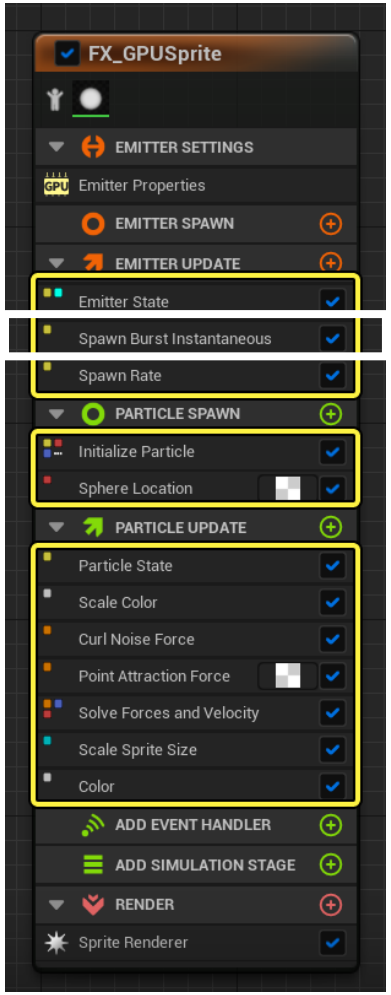
4주차 진행 · 모듈화

기존 방식 파티클 Update: 고정된 구조체 + 고정된 CS 하나



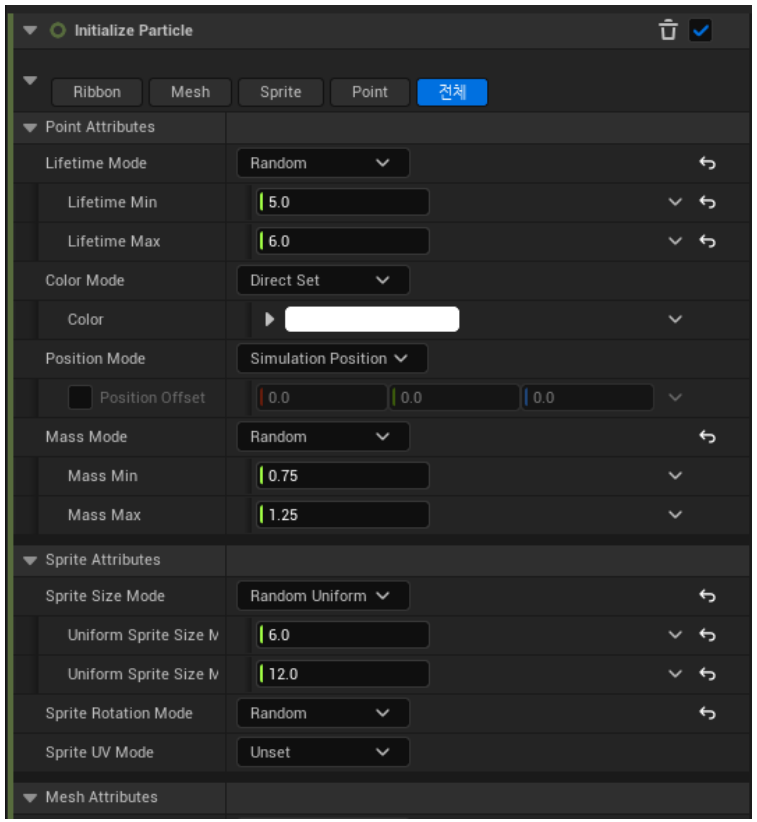
파티클의 업데이트를 모듈로 나눠 조립한다

1 모듈 리스트



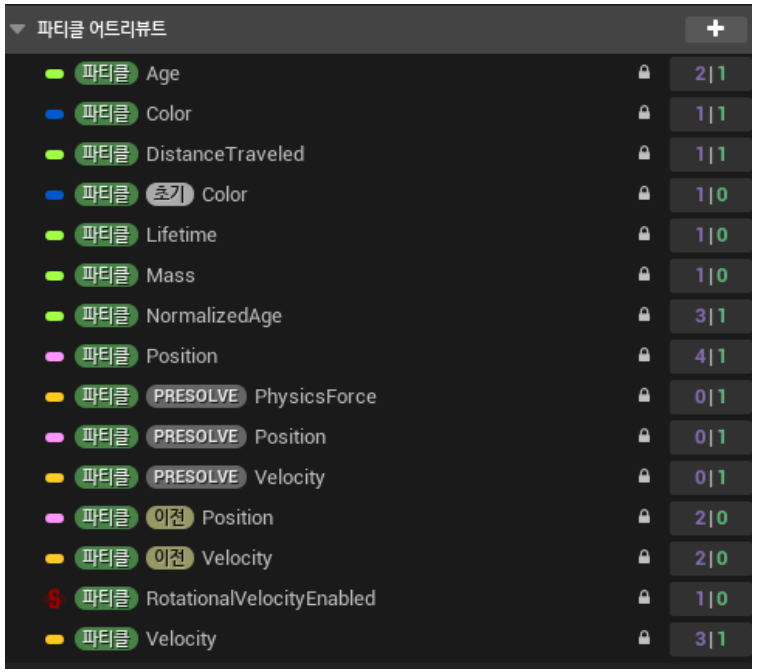
기능 하나 = 모듈 하나, 단계별로 쌓는다

2 모듈 파라미터



모듈마다 필요한 설정값

3 파티클 어트리뷰트



모듈이 읽고 쓰는 파티클 데이터

CPU에서의 모듈 모듈을 함수처럼 위에서 아래로 차례로 호출하면 된다

Emitter Update

SpawnRate	생성 개수

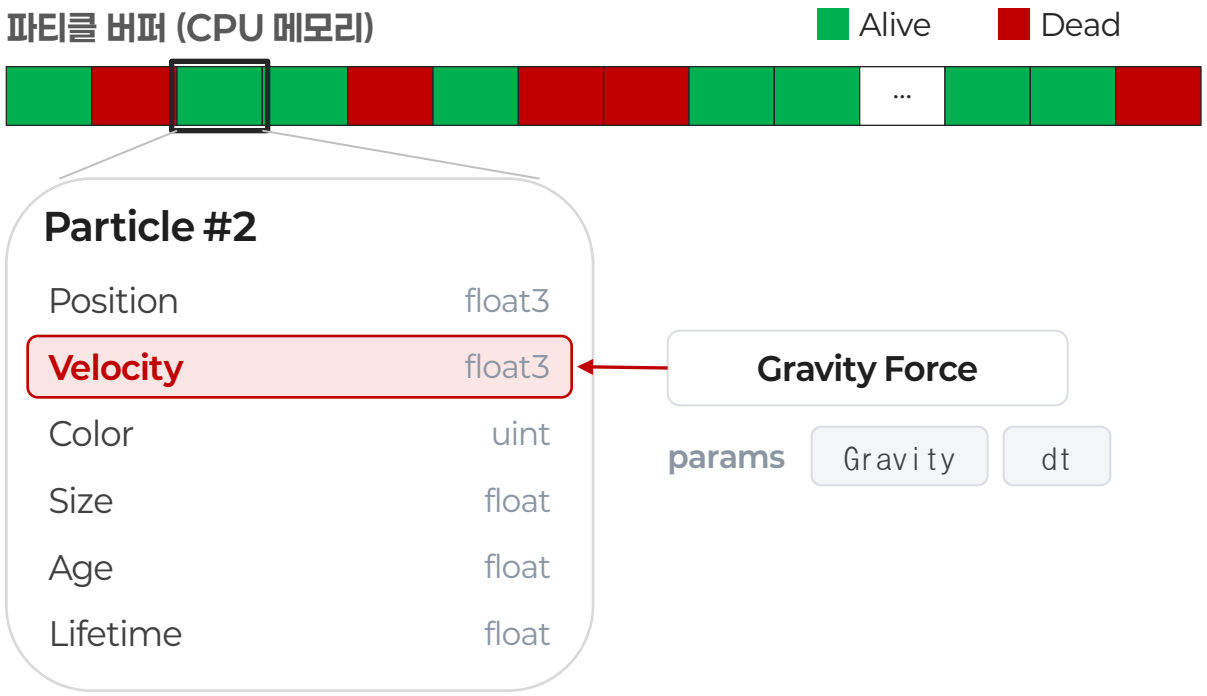
Particle Spawn

InitializeParticle	Color	Size	Lifetime
SphereLocation	Position		

Particle Update

ParticleState	Age
GravityForce	Velocity
SolveForceAndVelocity	Position Velocity

① 모듈을 순서대로 호출한다

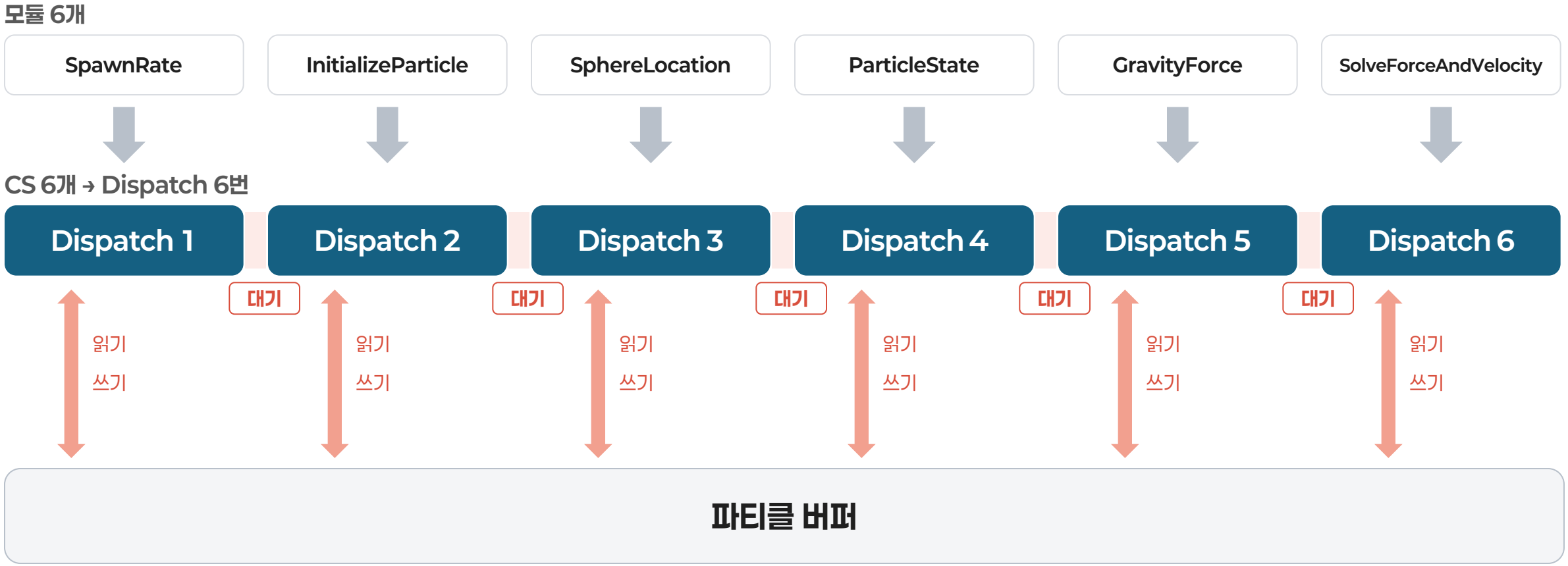



```
particles[2].Velocity += Gravity * dt;
```

② 메모리에서 바로 읽고 고친다

이미터마다 데이터 모양이 달라도 문제없다

GPU에서의 어려움 모듈마다 Dispatch하면 버퍼 왕복과 대기가 쌓인다

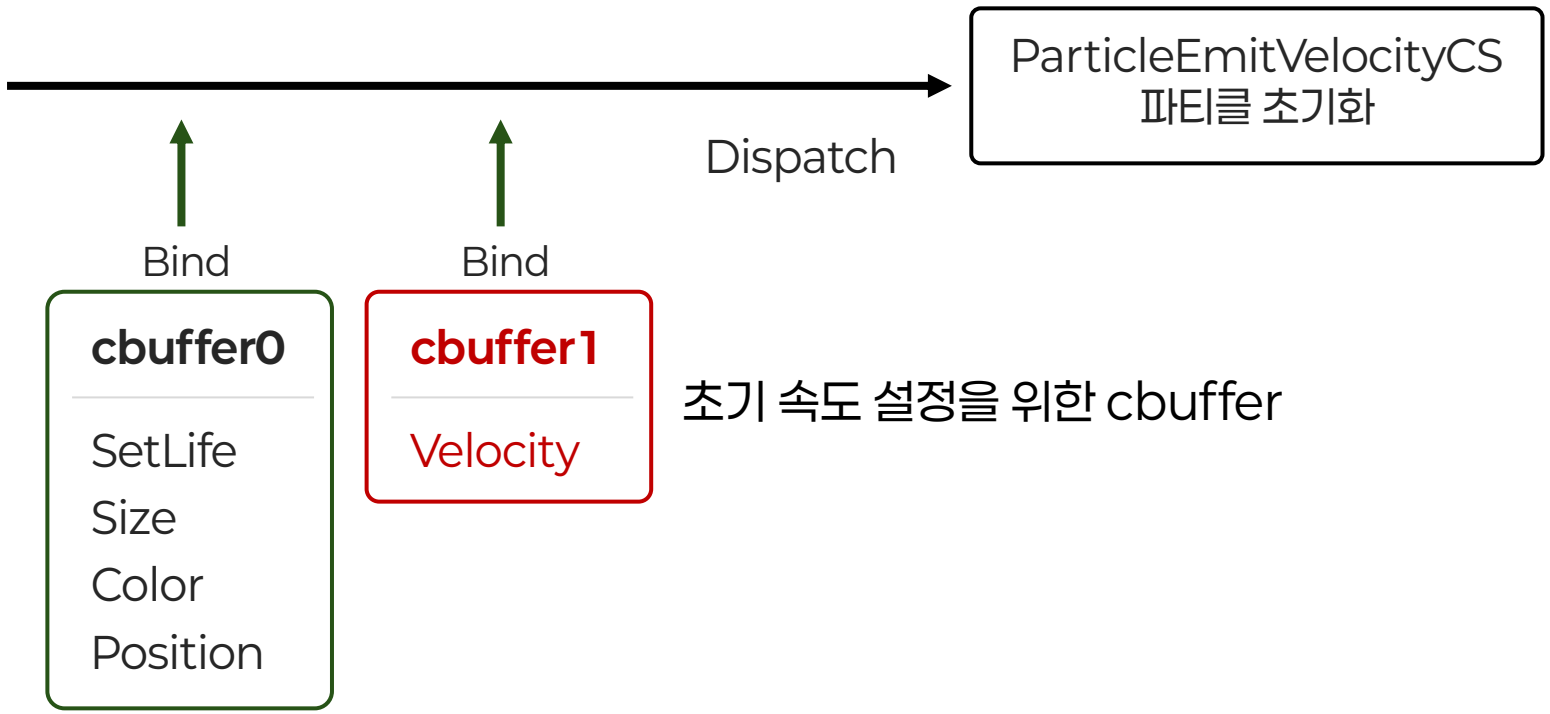


Dispatch마다 파티클 버퍼 전체를 다시 읽고 쓴다

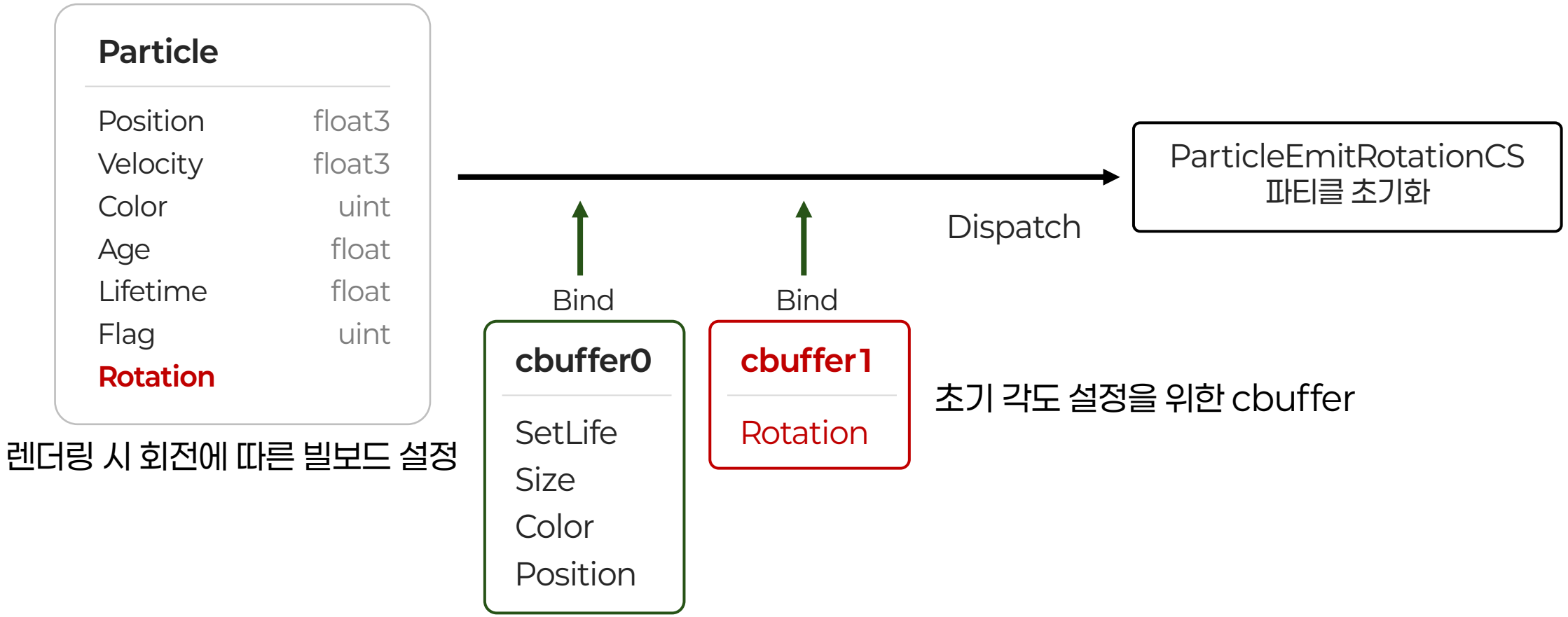
하나의 CS로 합친다면? ① 기능을 추가하면 → 새 cbuffer와 새 CS

Particle

Position	float3
Velocity	float3
Color	uint
Age	float
Lifetime	float
Flag	uint



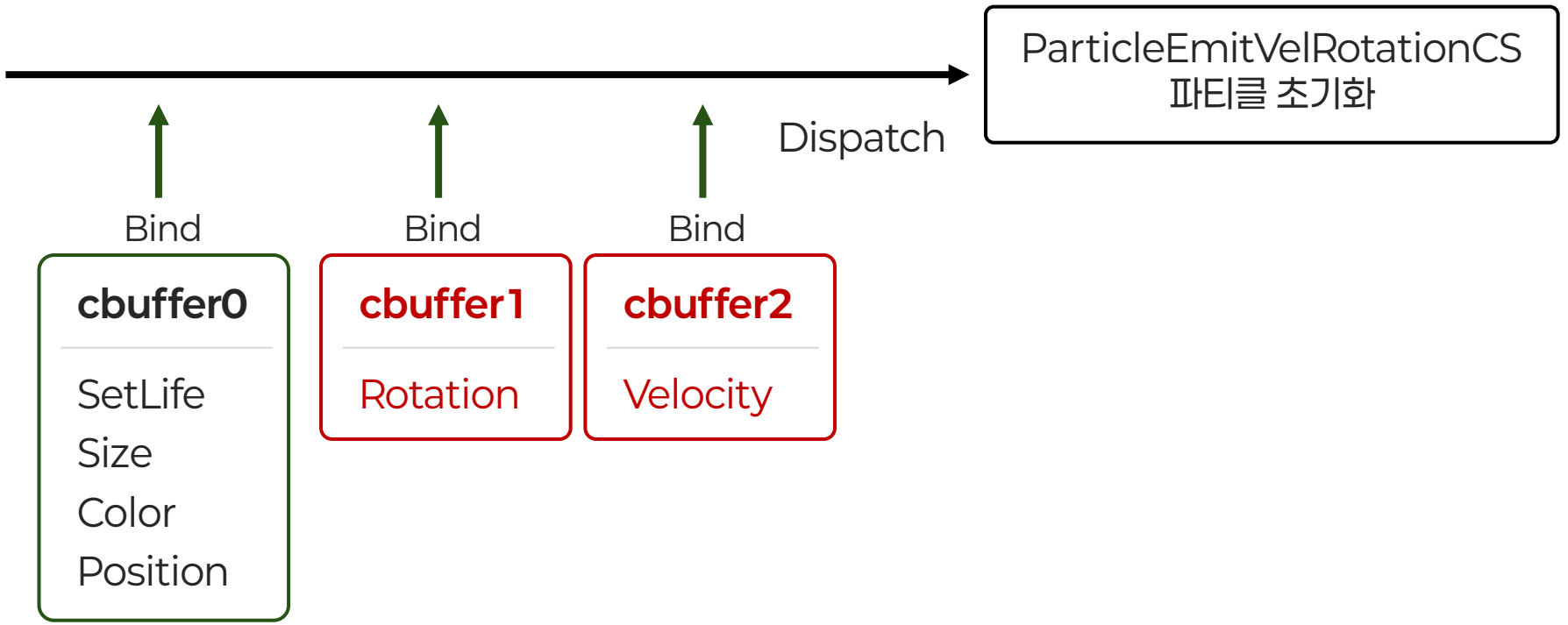
하나의 CS로 합친다면? ② 데이터를 추가하면 → 구조체와 렌더링까지 변경



하나의 CS로 합친다면? ③ 기능을 조합하면 → 조합마다 CS를 또 작성

Particle

Position	float3
Velocity	float3
Color	uint
Age	float
Lifetime	float
Flag	uint
Rotation	
...	



모듈마다 Dispatch

+ 조합은 자유롭다
− Dispatch마다 버퍼 왕복과 대기

VS

이미터마다의 모듈 조합을 하나의 CS로 합치기

+ Dispatch가 적어 빠르다
− 조합마다 CS를 직접 작성

도전 과제

1. 각 단계(Update/Emit)에서 어떤 데이터를 읽고 써야 하는지 어떻게 설정할 것인가?
2. 파티클마다 필요한 데이터가 다를 때, 파티클 구조는 어떻게 정해야 하는가?
3. 기능 조합마다 셰이더를 새로 짜지 않으면서, 단계마다 한 번의 Dispatch로 실행하려면 어떻게 해야 하는가?

모듈 정의 모듈이 실행 단계와 읽고 쓰는 데이터를 스스로 밝힌다

InitializeParticle

Stage

`ParticleSpawn`

reads

`OwnerPosition`

writes

Position | Color | Size | Lifetime | Age

params

`size` | `lifeTime` | `color`

+

GravityForce

writes — Velocity

→

파티클 구조체

- Position
- Velocity
- Color
- Size
- Age
- Lifetime

모듈들의 writes를 모으면 이 파티클에 사용되는 구조체가 된다

버퍼 구성 필요한 속성만으로 버퍼를 만들고, struct 없이 offset으로 읽는다

```
struct Particle
{
    float3 Position;
    float4 Color;
    float Size;
    float Age;
    float LifeTime;
};
```

문제

이 구조체는 어디서 정의하나?

· 이미터마다 모듈 조합이 달라 레이아웃도 매번 달라진다.
· CPP와 셰이더에 struct를 미리 써 둘 수 없다.

1 Byte Address Buffer

struct 없이 offset으로 읽고 쓴다

```
Position = LoadF3(buffer, OFS_Particles_Position, i);
Size     = LoadF1(buffer, OFS_Particles_Size,     i);
```

구조체 정의 없이 레이아웃 표만으로 접근

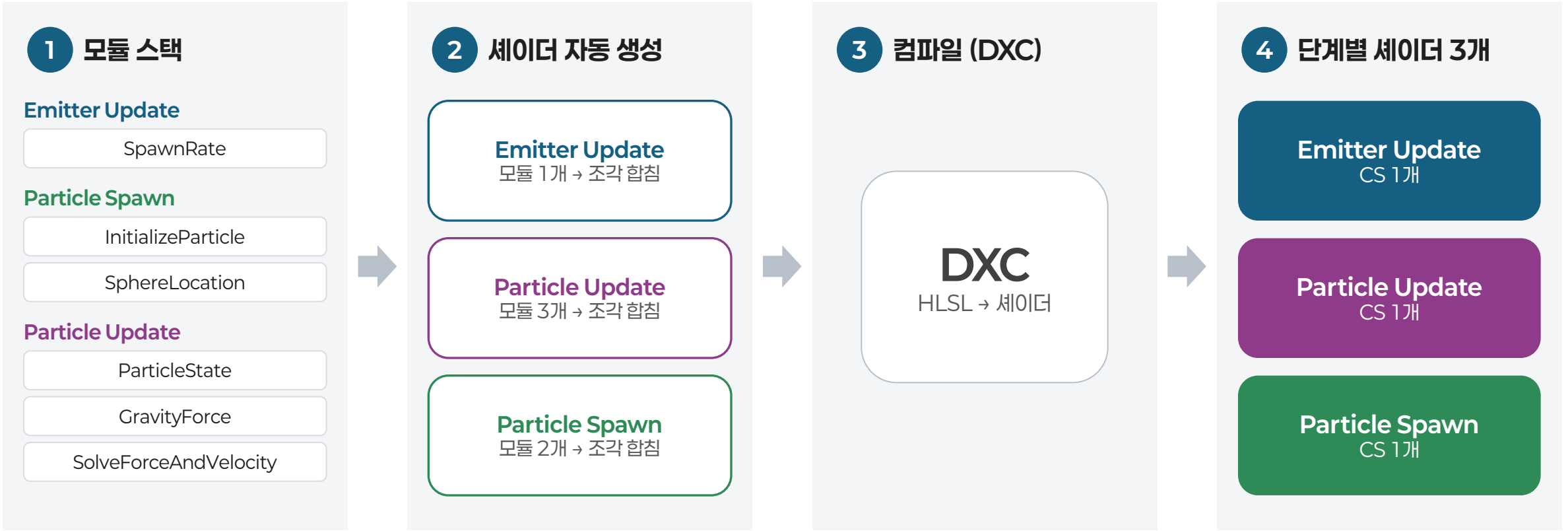
2 SoA 형태

예: Position(P), Size(S)만 쓰는 모듈



모듈은 자기가 쓰는 속성만 알면 된다

GPU 흐름 ① 준비 모듈 스택이 단계별 셰이더로 자동 생성된다



모듈 조합이 바뀌면 이 과정을 다시 거친다

사람이 셰이더를 직접 짜지 않는다

ParticleUpdateMain — 고정 틀

```
[numthreads(64, 1, 1)]
void ParticleUpdateMain(uint3 dtid : SV_DispatchThreadID)
{
    uint i = dtid.x;
    if (i >= Counters.Load(COUNTER_ALIVE_IN)) return;

    Particle p = LoadParticle(ParticlesIn, i);    // 읽기
    bool alive = true;
    ParticleUpdate(p, i, alive);                  // 모듈 조각 실행
    if (!alive) return;                           // 죽으면 쓰지 않음

    uint dst;
    Counters.InterlockedAdd(COUNTER_WRITE_CURSOR, 1, dst);
    StoreParticle(ParticlesOut, dst, p);          // 쓰기
}
```

LoadParticle 안쪽 11번의 offset으로 SoA 버퍼에서 읽는다

```
p.Position = LoadF3(b, OFS_Particles_Position, i);
p.Velocity = LoadF3(b, OFS_Particles_Velocity, i);
```

ParticleUpdate() — 모듈 조각을 순서대로 붙인 함수

```
void ParticleUpdate(inout Particle p, uint index, inout bool alive)
{
    M3 ParticleState
    p.Age += DeltaTime;
    float t = p.Age / p.Lifetime;
    if (PARAM(KillOnLifetime) != 0 && t >= 1.f) KILL();

    M4 GravityForce
    p.Acceleration += PARAM(Gravity) / p.Mass;

    M5 SolveForceAndVelocity
    p.Velocity += p.Acceleration * DeltaTime;
    p.Position += p.Velocity * DeltaTime;
    p.Acceleration = float3(0, 0, 0);
}
```

PARAM(x) = 그 모듈의 입력값 (편집기에서 바꾸는 값)

Emitter Update · Particle Spawn 함수도 같은 방식

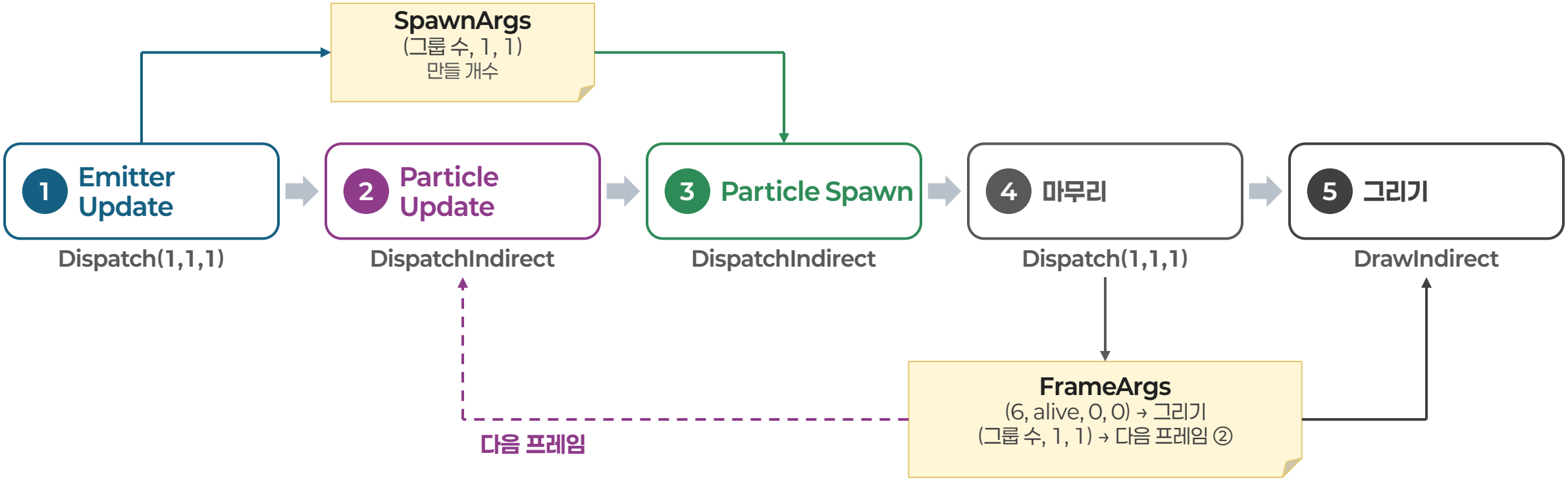
모듈 조합이 바뀌면 이 함수 안의 조각만 바뀐다 → 단계마다 함수 하나 = Dispatch 한 번

GPU 흐름 ② 매 프레임 단계별로 한 번씩만 Dispatch한다

CPU 각 단계의 실행 명령을 내린다

GPU

1. **Emitter Update** — 이번 프레임에 만들 개수 계산
   SpawnRate
2. **Particle Update** — 살아 있는 파티클 갱신, 수명 끝난 파티클 제거
   ParticleState · GravityForce · SolveForceAndVelocity
3. **Particle Spawn** — 새 파티클 생성
   InitializeParticle · SphereLocation
4. **마무리** — 살아 있는 개수 확정
5. **그리기** — 확정한 개수만큼 바로 그림


SpawnArgs
(그룹 수, 1, 1)
만들 개수
1 Emitter Update
Dispatch(1,1,1)
2 Particle Update
DispatchIndirect
3 Particle Spawn
DispatchIndirect
4 마무리
Dispatch(1,1,1)
5 그리기
DrawIndirect
다음 프레임
FrameArgs
(6, alive, 0, 0) → 그리기
(그룹 수, 1, 1) → 다음 프레임 ②

렌더링 셰이더 하나로 그리고, offset으로 필요한 데이터만 읽는다

1 렌더러가 읽을 칸을 밝힌다

Position ← Particles.Position
기본값 (0, 0, 0)

Color ← Particles.Color
기본값 (1, 1, 1, 1)

Size ← Particles.Size
기본값 0.1

모듈의 reads처럼, 렌더러도 읽을 데이터를 밝힌다

2 offset으로 바뀐다

Position	0
Color	12
Size	28

파티클 레이아웃에서 찾는다
cbuffer로 VS에 넘긴다

3 고정된 VS가 offset으로 읽는다

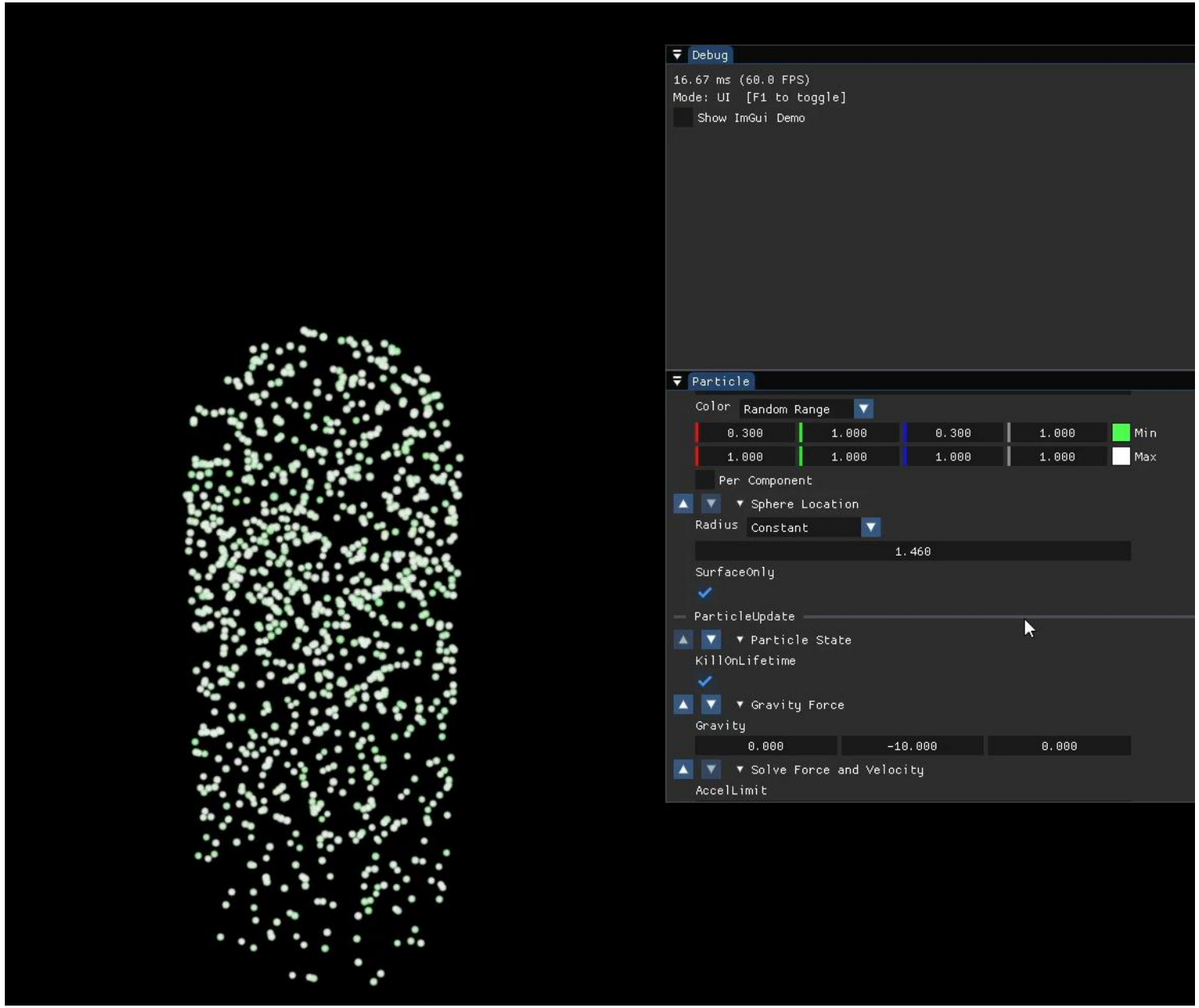
instanceId = DrawIndirect의 인스턴스 = 파티클 하나

```
uint idx = instanceId;   // 인스턴스 번호 = 파티클 번호

float3 position = (PositionOffset == NO_ATTRIBUTE) ? DefaultPosition
    : asfloat(ParticleData.Load3(ColumnAddress(PositionOffset, 12, idx)));
float4 color    = (ColorOffset == NO_ATTRIBUTE) ? DefaultColor
    : asfloat(ParticleData.Load4(ColumnAddress(ColorOffset, 16, idx)));
float  size     = (SizeOffset == NO_ATTRIBUTE) ? DefaultSize
    : asfloat(ParticleData.Load(ColumnAddress(SizeOffset, 4, idx)));
```

ColumnAddress = offset × Capacity + 번호 × 크기

렌더러가 읽을 칸을 offset으로 바꿔 넘기면, 셰이더 하나가 그 위치에서 읽는다



모듈 조합을 통해 만든 파티클

감사합니다

부록 InitializeParticle 모듈 정의 코드

```
ModuleDesc InitializeParticle(Input<float> size, Input<float> mass, Input<float> lifeTime, Input<float4> color)
{
ModuleDesc m;

m.name = "Initialize Particle";
m.stage = Stage::ParticleSpawn;
m.params =
{
     MakeParam<float>("Size", size).WithMin(0.f),
     MakeParam<float>("Mass", mass).WithMin(0.001f),
     MakeParam<float>("Lifetime", lifeTime).WithMin(0.f),
     MakeParam<float4>("Color", color).AsColor()
};
m.reads =
{
     {Engine::OwnerPosition, &Types::Float3}
};
m.writes =
{
     {Particles::Position, &Types::Float3},
     {Particles::Color, &Types::Float4},
     {Particles::Size, &Types::Float},
     {Particles::Lifetime, &Types::Float},
     {Particles::Age, &Types::Float},
     {Particles::Mass, &Types::Float}
};
m.hlsl = R"(
     p.Position = OwnerPosition;
     p.Mass     = PARAM(Mass);
     p.Color    = PARAM(Color);
     p.Size     = PARAM(Size);
     p.Lifetime = PARAM(Lifetime);
     p.Age      = 0.f;
)";
m.execute = [](ModuleContext& ctx)
{
     ...
};
return m;
}
```